

TradeMirror

Live Indicator & Bot Execution Architecture

Overview

TradeMirror's bot and live analytics system are built on a single shared intelligence pipeline. The same engine that powers the live Analytics panels evaluates trade conditions and executes orders. There is no separate bot pipeline and no duplicated indicator logic.

Core principle: *The Rust backend is the brain. The frontend is the display.*

All time-sensitive work happens in the Rust backend:

- Price ingestion from OANDA
- Candle management per timeframe
- Indicator calculation
- Condition evaluation
- Trade execution
- Telegram notification

The frontend receives push notifications from Rust and renders them. It never computes anything on the execution path.

Full Data Flow

Step 1 — OANDA Streaming Price Feed

A persistent WebSocket connection to OANDA receives tick-level price data for subscribed currency pairs. This is the entry point for all live data.

Step 2 — Price Tick Handler (Rust)

Each incoming tick is received by the Rust tick handler. It updates the current unclosed candle for every active timeframe simultaneously.

Step 3 — Candle Buffer (Rust, In-Memory)

Each pair and timeframe combination maintains its own rolling candle buffer in Rust memory. The buffer holds the last 200 closed candles plus the current live candle. When a new boundary is crossed (minute, 5-minute, hourly, etc.), the live candle is closed, appended to the buffer, and a new live candle begins.

Supported timeframes: M1, M5, M15, H1, H4, D, W

The candle buffer is held entirely in memory. No database read/write occurs on the live data path.

Step 4 — Indicator Engine (Rust, Incremental)

On each tick or candle close, the indicator engine receives the candle buffer and produces a structured `IndicatorSnapshot`. Indicators computed include:

Trend:

- EMA9, EMA20, EMA50, EMA100, EMA200
- ADX, DI+, DI-

Momentum:

- RSI9, RSI14
- MACD, MACD Signal, MACD Histogram
- CCI, Williams %R, ROC

Volatility:

- Bollinger Bands (upper, middle, lower)
- Keltner Channels (upper, middle, lower)
- ATR14
- Squeeze

The `IndicatorSnapshot` also includes pre-computed boolean state flags for instant condition evaluation:

- `macd_bullish_cross`
- `macd_bearish_cross`
- `rsi9_oversold` (`RSI9 < 30`)
- `rsi9_overbought` (`RSI9 > 70`)
- `rsi14_oversold` (`RSI14 < 30`)
- `rsi14_overbought` (`RSI14 > 70`)

Step 5 — Two Simultaneous Consumers

Consumer A — Condition Engine (Rust)

Evaluates strategy rules against the latest `IndicatorSnapshot`. When conditions are met, it acts immediately within the same Rust process — no inter-process communication, no round-trips.

Consumer B — Tauri Push Events (Frontend)

Pushes the `IndicatorSnapshot` to the Analytics UI. Panels re-render in real-time. No polling, no request/response round-trips.

Step 6 — When a Condition Fires

1. OANDA order is placed via direct API call (same Rust process)
2. Telegram notification sent via HTTP to AlphaAlert bot
3. Trade record written to SQLite for logging
4. Tauri event pushed to frontend for alert panel update

Why Execution Logic Lives in Rust

On a 1-minute timeframe, latency compounds quickly. Every millisecond removed from the path between signal detection and order submission matters.

Path A — Condition Check in Frontend (Slow)

Tick arrives in Rust	
Rust sends event to frontend	+1-5ms
Frontend receives and processes	+1-5ms
Frontend evaluates condition	+1ms
Frontend calls Rust to execute	+1-5ms
Rust calls OANDA API	+50-200ms (network floor)
Unnecessary overhead: 10-20ms	

Path B — Condition Check in Rust (Fast)

Tick arrives in Rust	
Rust updates indicators	<1ms
Rust evaluates condition	<1ms
Rust calls OANDA API	+50-200ms (network floor)
Unnecessary overhead: ~0ms	

The OANDA network call is the irreducible latency floor. Everything before it should be eliminated. On 1M candles, 10-20ms is the difference between acting on the candle that triggered the signal and acting on the next one.

Example Strategy: MACD + RSI Divergence

Rule 1 — Bullish Entry

- MACD bullish cross confirmed
- RSI14 below 30 (oversold)
- **Action: Place BUY order on OANDA**

Rule 2 — Bearish Entry

- MACD bearish cross confirmed
- RSI14 above 70 (overbought)
- **Action: Place SELL order on OANDA**

On condition fire: order placed, Telegram notification sent, trade logged. Rules are composable — multiple conditions across multiple timeframes can be combined. For example, H1 directional bias must be confirmed before a M5 entry signal is accepted.

Shared Pipeline

All systems consume the same candle buffer and indicator engine. There is never more than one indicator calculation per pair and timeframe.

System	Candle Buffer	Indicator Engine	Notes
--------	---------------	------------------	-------

System	Candle Buffer	Indicator Engine	Notes
Analytics Panels	Shared	Shared	Display only, receives push events
Alerts Engine	Shared	Shared	Condition monitoring, no execution
Bot Condition Engine	Shared	Shared	Evaluates rules, triggers orders
Trade Execution	—	—	Fires on condition trigger
Backtester	Separate (historical)	Shared logic	Replay mode, not live path

Build Order

Phase 1 — Live Analytics Foundation (Current)

5. OANDA streaming WebSocket connection in Rust
6. In-memory candle buffer per pair and timeframe
7. Indicator engine — recomputes on tick/close, produces IndicatorSnapshot
8. Tauri push events — streams IndicatorSnapshot to frontend
9. Analytics panels updated to subscribe to events and display live indicators

Phase 2 — Bot Execution (Next, After Analytics Validated)

10. Condition engine — evaluates strategy rules against IndicatorSnapshot
11. OANDA order execution on condition trigger
12. Telegram notification routing
13. Trade logging and position tracking

Phase 3 — Multi-Strategy and Risk (Later)

14. Rule composition across timeframes
 15. Risk engine — position sizing, max loss limits, drawdown protection
 16. Strategy management UI
-

What Does Not Change

- The existing daily SQLite pipeline remains untouched for historical replay
- The Analytics V3 UI layout and panel structure are preserved
- The existing OANDA daily sync continues for end-of-day persistence
- The existing Telegram bot (AlphaAlert) is reused as the notification router